Session 9: Logistic Regression

Complete Implementation Guide
From Regression to Classification
Building on Sessions 7 & 8: The Complete Machine Learning Foundation

Machine Intelligence Course - Aligned with Instructor's Code

IMPORTANT!

This document follows the EXACT structure from the course file:

- `Session9_Logistic_Regression_Solution.py` - INSTRUCTOR'S SOLUTION

All code shown is from the instructor's file. No additions, no modifications!

Contents

1 The Complete Journey: Sessions 7, 8, and 9 **2**

2 From Regression to Classification **3**
2.1 The Fundamental Difference . . . 3
2.2 Binary Classification Setup . . . 4

3 Exercise 1A: The Sigmoid Function **4**
3.1 Definition and Properties . . . 4
3.2 Implementation . . . 5

4 Logistic Regression Model **6**
4.1 The Complete Model . . . 6
4.2 Decision Boundaries . . . 8

5 Exercise 2: Cost Function for Logistic Regression **8**
5.1 Why Not Squared Error? . . . 8
5.2 Binary Cross-Entropy Loss . . . 9
5.3 Total Cost Function . . . 10

6 Exercise 3: Gradient Descent for Logistic Regression **11**
6.1 The Gradient Formulas . . . 11
6.2 Gradient Descent Algorithm . . . 13

7 Exercise 4: Making Predictions **15**
7.1 From Probabilities to Decisions . . . 15

8 Exercise 5: Regularization **16**
8.1 The Regularization Term . . . 16
8.2 Regularized Gradients . . . 18

9 Summary: Key Concepts **19**
9.1 Complete Comparison: Sessions 7, 8, 9 . . . 19
9.2 Key Formulas to Memorize . . . 20
9.3 Master These Concepts . . . 22

10 Looking Ahead **23**
10.1 Practical Tips . . . 23

11 Conclusion: The Complete Picture **25**

12 What You Have ACTUALLY Learned **26**
12.1 Deep Understanding of Core Concepts . . . 27
12.1.1 1. The Sigmoid Function . . . 27
12.1.2 2. Cross-Entropy Loss . . . 28
12.1.3 3. Decision Boundaries . . . 29
12.2 Regularization: The Critical Addition . . . 29
12.2.1 1. Why Regularization for Classification? . . . 30
12.2.2 2. Regularized Cost Function . . . 30
12.2.3 3. Regularized Gradients . . . 31
12.2.4 4. Choosing λ . . . 32
12.2.5 5. L2 vs L1 Regularization . . . 33
12.3 Connection to Neural Networks . . . 34
12.4 Practical Lessons from Implementation . . . 35
12.5 Real-World Applications . . . 36
12.6 What You Can Now Explain . . . 36
12.7 The Complete Machine Learning Foundation . . . 38
12.8 Key Takeaways . . . 39

1 The Complete Journey: Sessions 7, 8, and 9

RED THREAD: The Journey from Session 7 → 8 → 9

Session 7: Linear Regression (Single Feature)

- **Task**: Predict house prices from size
- **Output**: Continuous number (any real value)
- **Model**: $f_{w,b}(x) = wx + b$
- **Concepts**: Gradient descent, cost function, vectorization

Session 8: Multiple Linear Regression

- **Task**: Predict house prices from size, bedrooms, floors, age
- **Output**: Continuous number (any real value)
- **Model**: $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$
- **New concepts**: Multiple features, feature scaling, polynomial features, overfitting

Session 9: Logistic Regression (Classification!)

- **Task**: Classify students (admitted/not admitted)
- **Output**: Probability between 0 and 1 (discrete decision)
- **Model**: $f_{\vec{w},b}(\vec{x}) = \frac{1}{1+e^{-(\vec{w}\cdot\vec{x}+b)}}$
- **New concepts**: Sigmoid function, binary cross-entropy, decision boundaries, classification metrics

The Big Shift: From predicting "how much" to predicting "yes or no"!
What stays the same:

- Gradient descent optimization
- Cost function that we minimize
- Feature scaling importance
- Regularization to prevent overfitting
- Vectorized implementation for speed

What's new:

- Sigmoid function squashes output to [0,1]
- Binary cross-entropy loss instead of squared error
- Decision boundaries instead of regression lines
- Classification metrics (accuracy) instead of continuous error

2 From Regression to Classification

2.1 The Fundamental Difference

Regression vs Classification

REGRESSION (Sessions 7 & 8):

- **Question**: "How much?" or "What value?"
- **Examples**:
  - Predict house price: \$250,000 (any number)
  - Predict temperature: 23.5°C
  - Predict stock price: \$142.75
- **Output range**: $(-\infty, \infty)$ (any real number)
- **Model**: Linear combination of features

CLASSIFICATION (Session 9):

- **Question**: "Which category?" or "Yes or no?"
- **Examples**:
  - Is this email spam? (Yes/No)
  - Will student be admitted? (Yes/No)
  - Is tumor malignant? (Yes/No)
- **Output range**: $[0, 1]$ (probability)
- **Model**: Linear combination + Sigmoid function

The key insight: Linear regression predicts continuous values. Classification predicts discrete categories!

IMPORTANT!

Why NOT use Linear Regression for Classification?
Problem 1: Linear regression outputs can be ANY value

- $f(\vec{x})$ could be -5, 0.3, 2.7, 100, etc.
- We need output between 0 and 1 for probability

Problem 2: Linear regression doesn't fit classification data well

- Straight line doesn't capture the S-shaped transition
- Sensitive to outliers (one extreme point shifts everything)
- Treats 0 and 1 as continuous values (they're not!)

Solution: Use logistic regression with sigmoid function!

2.2 Binary Classification Setup

Binary Classification

Definition: Classify input into ONE of TWO classes
Standard notation:

- $y \in \{0, 1\}$ (only two possible values!)
- $y = 1$: Positive class (e.g., "admitted", "spam", "malignant")
- $y = 0$: Negative class (e.g., "not admitted", "not spam", "benign")

Training data: $\{(\vec{x}^{(1)}, y^{(1)}), (\vec{x}^{(2)}, y^{(2)}), \ldots, (\vec{x}^{(m)}, y^{(m)})\}$
Each $y^{(i)}$ is either 0 or 1 (nothing in between!)

3 Exercise 1A: The Sigmoid Function

3.1 Definition and Properties

RED THREAD: The Journey from Session 7 → 8 → 9

From Linear Regression:

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b \quad \text{(can be any number)}$$

To Logistic Regression:

$$f_{\vec{w},b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}} \quad \text{(always between 0 and 1)}$$

The sigmoid function $g(z)$ is the magic that constrains output to [0,1]!

Sigmoid Function (Logistic Function)

Formula:

$$g(z) = \frac{1}{1 + e^{-z}}$$

Key properties:

- **Range**: $g(z) \in (0, 1)$ for all $z \in \mathbb{R}$
- **Midpoint**: $g(0) = 0.5$
- **S-shaped curve**: Smooth transition from 0 to 1
- **Asymptotes**:
  - As $z \to \infty$: $g(z) \to 1$
  - As $z \to -\infty$: $g(z) \to 0$
- **Symmetry**: $g(-z) = 1 - g(z)$

Behavior:

Input z	Output $g(z)$	Interpretation
$z = -10$	≈ 0.00005	Very confident $y = 0$
$z = -5$	≈ 0.007	Strong evidence for $y = 0$
$z = 0$	0.5	Uncertain (50-50)
$z = 5$	≈ 0.993	Strong evidence for $y = 1$
$z = 10$	≈ 0.99995	Very confident $y = 1$

3.2 Implementation

sigmoid - FROM INSTRUCTOR'S CODE - INSTRUCTOR'S CODE

File: Session9_Logistic_Regression_Solution.py (lines 283-299)

```python
# UNQ_C1
# GRADED FUNCTION: sigmoid

def sigmoid(z):
    """
    Compute the sigmoid of z

    Args:
        z (ndarray): A scalar, numpy array of any size.

    Returns:
        g (ndarray): sigmoid(z), with the same shape as z

    """
    # START CODE HERE
    # Compute sigmoid:  1 / (1 + exp(-z))
    g = 1/(1 + np.exp(-z))
    # END SOLUTION
    # Return sigmoid values (same shape as input)
    return g
```

```python
# Test the sigmoid function at zero
print ("sigmoid(0) = " + str(sigmoid(0)))
```

How it works:

- `np.exp(-z)` computes e^{-z}
- Add 1: $1 + e^{-z}$
- Take reciprocal: $\frac{1}{1+e^{-z}}$
- Works for scalars AND arrays (vectorized!)

Understanding the Sigmoid

What does it do? "Squashes" any real number into (0,1)
Example calculations:

- $g(0) = \frac{1}{1+e^0} = \frac{1}{1+1} = 0.5$
- $g(2) = \frac{1}{1+e^{-2}} \approx \frac{1}{1.135} \approx 0.88$
- $g(-3) = \frac{1}{1+e^3} \approx \frac{1}{21.09} \approx 0.047$

Plain English:

- Large positive $z \to$ sigmoid close to 1 $\to$ predict $y = 1$
- Large negative $z \to$ sigmoid close to 0 $\to$ predict $y = 0$
- z near 0 $\to$ sigmoid near 0.5 $\to$ uncertain!

4 Logistic Regression Model

4.1 The Complete Model

Logistic Regression Model

Formula:

$$f_{\vec{w},b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b) = \frac{1}{1 + e^{-(\vec{w}\cdot\vec{x}+b)}}$$

Step-by-step breakdown:

1. **Linear combination**: $z = \vec{w} \cdot \vec{x} + b$ (same as linear regression!)
2. **Apply sigmoid**: $g(z) = \frac{1}{1+e^{-z}}$ (new!)
3. **Output**: Probability between 0 and 1

Output interpretation:

$$f_{\vec{w},b}(\vec{x}) = P(y = 1 \mid \vec{x}; \vec{w}, b)$$

The probability that $y = 1$ given input $\vec{x}$ and parameters $\vec{w}, b$
Making a decision:

- If $f_{\vec{w},b}(\vec{x}) \geq 0.5$: Predict $\hat{y} = 1$
- If $f_{\vec{w},b}(\vec{x}) < 0.5$: Predict $\hat{y} = 0$

compute_model_output - FROM INSTRUCTOR'S CODE - INSTRUCTOR'S CODE

File: Session9_Logistic_Regression_Solution.py (lines 339-350)

```python
def compute_model_output(x, w, b):
    """
    Computes the prediction of a linear model
    Args:
      x (ndarray (m,)): Data, m examples
      w,b (scalar)    : model parameters
    Returns
      y (ndarray (m,)): target values
    """
    # Compute linear combination:  w dot x + b
    # Apply sigmoid to get probability
    f_wb=sigmoid(np.dot(x,w) + b)
    # Return probability between 0 and 1
    return f_wb
# Example inputs for testing
x = [[1, 0]]
w = [1, 10]
b = 0
# Compute and print model output
print ("f(x,w,b) = " + str(compute_model_output(x,w,b)))
```

How it works:

1. `np.dot(x, w)` computes $\vec{w} \cdot \vec{x}$
2. Add bias: `+ b`
3. Apply sigmoid to get probability
4. Return value in [0,1]

4.2 Decision Boundaries

Decision Boundary

Definition: The boundary where the model is exactly 50-50 uncertain
Mathematical definition: Since $g(z) = 0.5$ when $z = 0$, the decision boundary is:

$$\vec{w} \cdot \vec{x} + b = 0$$

Prediction rules:

- If $\vec{w} \cdot \vec{x} + b \geq 0$: Predict $\hat{y} = 1$
- If $\vec{w} \cdot \vec{x} + b < 0$: Predict $\hat{y} = 0$

For 2 features (x_1, x_2):

- Decision boundary is a line: $w_1 x_1 + w_2 x_2 + b = 0$
- One side $\rightarrow$ predict $y = 1$
- Other side $\rightarrow$ predict $y = 0$

Plain English: The decision boundary divides the feature space into "yes" and "no" regions!

Example: Student Admission

Suppose: Two exam scores (x_1, x_2) predict admission
Learned parameters: $w_1 = 1$, $w_2 = 1$, $b = -3$
Decision boundary: $x_1 + x_2 - 3 = 0 \Rightarrow x_2 = 3 - x_1$
Predictions:

- Student with scores (2, 2): $z = 2 + 2 - 3 = 1 > 0 \rightarrow$ Admit!
- Student with scores (1, 1): $z = 1 + 1 - 3 = -1 < 0 \rightarrow$ Reject
- Student with scores (1.5, 1.5): $z = 0 \rightarrow$ Uncertain (50-50)

Visual: The line $x_1 + x_2 = 3$ separates admitted from rejected students!

5 Exercise 2: Cost Function for Logistic Regression

5.1 Why Not Squared Error?

IMPORTANT!

DON'T use squared error for logistic regression!
Linear regression cost:

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})^2$$

Problems when f includes sigmoid:

- Cost function becomes NON-CONVEX (wavy, multiple bumps)

- Gradient descent gets stuck in local minima
- No guarantee of finding global minimum
- Convergence is unreliable

Solution: Use binary cross-entropy loss (log loss)!

5.2 Binary Cross-Entropy Loss

Loss for Single Example

Loss function:

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(f_{\vec{w},b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

Compact form (works for both cases):

$$L = -y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))$$

Why it works:

- When $y^{(i)} = 1$: $(1 - y^{(i)}) = 0$, second term vanishes
- When $y^{(i)} = 0$: $y^{(i)} = 0$, first term vanishes

Understanding the Loss

Case 1: Actual label $y = 1$

- Loss $= -\log(f_{\vec{w},b}(\vec{x}))$
- If model predicts $f = 1$ (confident correct): Loss $= -\log(1) = 0$ ✓
- If model predicts $f = 0.5$: Loss $= -\log(0.5) \approx 0.69$
- If model predicts $f \to 0$ (confident wrong): Loss $\to \infty$ ×××

Case 2: Actual label $y = 0$

- Loss $= -\log(1 - f_{\vec{w},b}(\vec{x}))$
- If model predicts $f = 0$ (confident correct): Loss $= -\log(1) = 0$ ✓
- If model predicts $f = 0.5$: Loss $= -\log(0.5) \approx 0.69$
- If model predicts $f \to 1$ (confident wrong): Loss $\to \infty$ ×××

Key insight: The loss heavily penalizes confident wrong predictions!

5.3 Total Cost Function

Logistic Regression Cost Function

Average loss over all examples:

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)})$$

Expanded form:

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) \right]$$

where $f_{\vec{w},b}(\vec{x}^{(i)}) = \frac{1}{1+e^{-(\vec{w}\cdot\vec{x}^{(i)}+b)}}$

Properties:

- **Convex**: Has single global minimum (gradient descent works!)
- **Smooth**: Differentiable everywhere
- **Interpretable**: Measures how wrong predictions are

Also called: Binary cross-entropy, log loss

compute_cost - FROM INSTRUCTOR'S CODE - INSTRUCTOR'S CODE

File: Session9_Logistic_Regression_Solution.py (lines 434-456)

```python
def compute_cost(X, y, w, b, lambda_= 1):
    """
    Computes the cost over all examples
    Args:
      X : (ndarray Shape (m,n)) data, m examples by n features
      y : (array_like Shape (m,)) target value
      w : (array_like Shape (n,)) Values of parameters of the model
      b : scalar Values of bias parameter of the model
      lambda_: unused placeholder
    Returns:
      total_cost: (scalar)         cost
    """
    # Get number of examples and features
    m, n = X.shape
    # START CODE HERE
    # Compute linear combination and clip for stability
    logit = np.clip(np.dot(X, w) + b, -10, 10)
    # Apply sigmoid to get probabilities
    f_w = sigmoid(logit)
    # Compute binary cross-entropy loss
    total_cost = -np.mean(y*np.log(f_w) + (1-y)*np.log(1-f_w))
    # END CODE HERE
    # Return average cost over all examples
    return total_cost
# Get dimensions of training data
m, n = X_train.shape
# Initialize weights to zero
```

```
initial_w = np.zeros(n)
# Initialize bias to zero
initial_b = 0.
# Compute initial cost
cost = compute_cost(X_train, y_train, initial_w, initial_b)
print('Cost at initial w (zeros): {:.3f}'.format(cost))
```

How it works:

1. `np.dot(X, w) + b`: Compute z for all examples
2. `np.clip(..., -10, 10)`: Prevent overflow in sigmoid
3. `sigmoid(logit)`: Get probabilities
4. `y*np.log(f_w)`: First term of loss
5. `(1-y)*np.log(1-f_w)`: Second term of loss
6. `-np.mean(...)`: Average and negate

Note: `np.clip` prevents numerical issues with very large/small values!

6 Exercise 3: Gradient Descent for Logistic Regression

6.1 The Gradient Formulas

RED THREAD: The Journey from Session 7 → 8 → 9

Surprising fact: The gradient formulas look IDENTICAL to linear regression!
Gradient for weights:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})x_j^{(i)}$$

Gradient for bias:

$$\frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

Key difference: The definition of $f_{\vec{w},b}(\vec{x})$ is different!

- Linear regression: $f(\vec{x}) = \vec{w}\cdot\vec{x} + b$
- Logistic regression: $f(\vec{x}) = \frac{1}{1+e^{-(\vec{w}\cdot\vec{x}+b)}}$

Same formula, different model!

compute_gradient - FROM INSTRUCTOR'S CODE - INSTRUCTOR'S CODE

File: Session9_Logistic_Regression_Solution.py (lines 504-532)

```
# UNQ_C3
# GRADED FUNCTION: compute_gradient
def compute_gradient(X, y, w, b, lambda_=None):
    """
    Computes the gradient for logistic regression
```

```python

    Args:
      X : (ndarray Shape (m,n)) variable such as house size
      y : (array_like Shape (m,1)) actual value
      w : (array_like Shape (n,1)) values of parameters of the model
      b : (scalar)                 value of parameter of the model
      lambda_: unused placeholder.
    Returns
      dj_dw: (array_like Shape (n,1)) The gradient of the cost w.r.t. the parameters w.
      dj_db: (scalar)                 The gradient of the cost w.r.t. the parameter b.
    """
    # Get number of examples and features
    m, n = X.shape
    # Initialize gradient vector for weights
    dj_dw = np.zeros(w.shape)
    # Initialize gradient scalar for bias
    dj_db = 0.
    # START CODE HERE
    # Compute predictions using sigmoid
    f_w = sigmoid(np.dot(X, w) + b)
    # Compute prediction errors
    err = (f_w - y)
    # Weight gradient:  X transpose times errors, averaged
    dj_dw = (1/m)*np.dot(X.T, err)
    # Bias gradient:  sum of errors, averaged
    dj_db = (1/m)*np.sum(err)
    # END CODE HERE
    # Return both gradients
    return dj_db, dj_dw
# Initialize weights to zeros
initial_w = np.zeros(n)
# Initialize bias to zero
initial_b = 0.
# Compute gradients at initial parameters
dj_db, dj_dw = compute_gradient(X_train, y_train, initial_w, initial_b)
print(f'dj_db at initial w (zeros):{dj_db}' )
print(f'dj_dw at initial w (zeros):{dj_dw.tolist()}' )
```

How it works (fully vectorized!):

1. `f_w = sigmoid(np.dot(X, w) + b)`: Predictions for ALL examples
2. `err = f_w - y`: Errors for ALL examples
3. `dj_dw = (1/m)*np.dot(X.T, err)`: Weight gradients (vectorized!)
4. `dj_db = (1/m)*np.sum(err)`: Bias gradient

Matrix magic:

- `X.T @ err` computes $\sum_{i=1}^{m}(f(\vec{x}^{(i)}) - y^{(i)})\vec{x}^{(i)}$
- All in one operation! No loops!

6.2 Gradient Descent Algorithm

Complete Gradient Descent

Algorithm:

$$\text{repeat until convergence}\{$$
$$w_j = w_j - \alpha \frac{\partial J}{\partial w_j} \quad \text{for } j = 1, \ldots, n$$
$$b = b - \alpha \frac{\partial J}{\partial b}$$
$$\}$$

In code:

- Compute gradients: `dj_dw, dj_db = compute_gradient(X, y, w, b)`
- Update weights: `w = w - alpha * dj_dw`
- Update bias: `b = b - alpha * dj_db`
- Repeat for many iterations

Important: All updates must be simultaneous (use temporary variables)!

gradient_descent_val - FROM INSTRUCTOR'S CODE - INSTRUCTOR'S CODE

File: Session9_Logistic_Regression_Solution.py (lines 557-603)
Note: This version splits data into train/validation and tracks accuracy!

```python
def gradient_descent_val(X, y, w_in, b_in, cost_function,
                         gradient_function, alpha, num_iters, lambda_):
    # Split the data into train and validation
    # Set random seed for reproducibility
    np.random.seed(123)
    # Generate random permutation
    perm = np.random.permutation(X.shape[0])
    # Shuffle features and labels
    X = X[perm,:]
    y = y[perm]
    # Split:  66 percent for training
    size_train = int(0.66*X.shape[0])
    # Create training set (features)
    X_split_train = X[0:size_train,:]
    # Create training set (labels)
    y_split_train = y[0:size_train]
    # Create validation set (features)
    X_split_val = X[size_train:,:]
    # Create validation set (labels)
    y_split_val = y[size_train:]
    # Initialize tracking lists
    # Track training cost
    J_history_train = []
    # Track training accuracy
    J_history_train_acc = []
```

```python
    # Track validation accuracy
    J_history_test = []
    # Deep copy initial weights
    w = copy.deepcopy(w_in)
    # Copy initial bias
    b = b_in
    # Main training loop
    for i in range(num_iters):
        # Calculate gradients on training data
        dj_db, dj_dw = gradient_function(X_split_train, y_split_train,
                                         w, b, lambda_)
        # Update weights via gradient descent
        w = w - alpha * dj_dw
        # Update bias via gradient descent
        b = b - alpha * dj_db
        # Save metrics every iteration
        if i<100000:
            # Compute and save training cost
            J_history_train.append(cost_function(X_split_train,
                                                 y_split_train, w, b))
            # Compute and save training accuracy
            J_history_train_acc.append(accuracy(X_split_train,
                                                y_split_train, w, b))
            # Compute and save validation accuracy
            J_history_test.append(accuracy(X_split_val,
                                           y_split_val, w, b))
        # Print progress periodically
        if i% math.ceil(num_iters / 50) == 0:
            print(f"Iteration {i:4d}: Training Cost {J_history_train
                [-1]:8.2f}, Train Accuracy {J_history_train_acc[-1]:8.2f
                }, Test Accuracy {J_history_test[-1]:8.2f}   ")
    # Plot training curves
    # Create iteration array for x-axis
    it = np.arange(0,num_iters,1)
    # Plot training cost over time
    plt.plot(it, J_history_train, label="Training Cost")
    # Plot training accuracy over time
    plt.plot(it, J_history_train_acc, label="Training Accuracy")
    # Plot validation accuracy over time
    plt.plot(it, J_history_test, label = "Test Accuracy")
    plt.legend()
    plt.show()
    # Return learned parameters and histories
    return w, b, J_history_train, J_history_test
```

What's special:

- Automatically splits data (66% train, 34% validation)
- Tracks BOTH training cost AND accuracy
- Tracks validation accuracy (detects overfitting!)
- Plots all metrics for visualization

7 Exercise 4: Making Predictions

7.1 From Probabilities to Decisions

Prediction Process

Step 1: Compute probability

$$f_{\vec{w},b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w}\cdot\vec{x}+b)}}$$

Step 2: Apply threshold

- If $f_{\vec{w},b}(\vec{x}) \geq 0.5$: Predict $\hat{y} = 1$
- If $f_{\vec{w},b}(\vec{x}) < 0.5$: Predict $\hat{y} = 0$

Output: Binary decision (0 or 1)

predict - FROM INSTRUCTOR'S CODE - INSTRUCTOR'S CODE

File: Session9_Logistic_Regression_Solution.py (lines 679-699)

```python
# UNQ_C4
# GRADED FUNCTION: predict

def predict(X, w, b):
    """
    Predict whether the label is 0 or 1 using learned logistic
    regression parameters w

    Args:
    X : (ndarray Shape (m, n))
    w : (array_like Shape (n,))      Parameters of the model
    b : (scalar, float)              Parameter of the model

    Returns:
    p: (ndarray (m,1))
        The predictions for X using a threshold at 0.5
    """
    # START CODE HERE
    # Compute probabilities using sigmoid
    f_w = sigmoid(np.dot(X, w) + b)
    # Apply threshold:  predict 1 if prob >= 0.5
    p = (f_w >=0.5)
    # END CODE HERE
    # Return binary predictions (0 or 1)
    return p
# Make predictions on training data
p = predict(X_train, w, b)
# Compute and print training accuracy
print('Train Accuracy: %f'%(np.mean(p == y_train) * 100))
```

How it works:

1. `f_w = sigmoid(...)`: Compute probabilities
2. `p = (f_w >= 0.5)`: Apply threshold (returns boolean)

3. Boolean True/False becomes 1/0 automatically!
4. `np.mean(p == y_train)`: Accuracy (fraction correct)

8 Exercise 5: Regularization

8.1 The Regularization Term

RED THREAD: The Journey from Session 7 → 8 → 9

From Sessions 7-8: We learned about overfitting
Solution: Regularization - penalize large weights
Same concept in Session 9:

- Add penalty term to cost function
- Shrink weights toward zero
- Keep all features, but reduce their impact
- Prevents overfitting in classification too!

Regularized Cost Function

Without regularization:

$$J(\vec{w}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1-y^{(i)})\log(1-f_{\vec{w},b}(\vec{x}^{(i)}))\right]$$

With regularization:

$$J(\vec{w}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1-y^{(i)})\log(1-f_{\vec{w},b}(\vec{x}^{(i)}))\right] + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$$

Regularization term: $\frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$

- λ: Regularization parameter (controls strength)
- $\lambda = 0$: No regularization
- λ large: Strong regularization (simpler model)
- Bias b is NOT regularized!

compute_cost_reg - FROM INSTRUCTOR'S CODE - INSTRUCTOR'S CODE

File: Session9_Logistic_Regression_Solution.py (lines 746-774)

```python
# UNQ_C5
def compute_cost_reg(X, y, w, b, lambda_ = 1):
    """
    Computes the cost over all examples
    Args:
      X : (array_like Shape (m,n)) data, m examples by n features
```

```
      y : (array_like Shape (m,)) target value
      w : (array_like Shape (n,)) Values of parameters of the model
      b : (array_like Shape (n,)) Values of bias parameter of the model
      lambda_ : (scalar, float)    Controls amount of regularization
    Returns:
      total_cost: (scalar)         cost
    """
    # Get number of examples and features
    m, n = X.shape
    # Compute base cost without regularization
    cost_without_reg = compute_cost(X, y, w, b)
    # START CODE HERE
    # Calculate regularization term:  (lambda/2m) * sum(w**2)
    reg_cost =  (lambda_/(2*m)) * np.sum(np.square(w))
    # END CODE HERE
    # Add regularization to base cost
    total_cost = cost_without_reg +  reg_cost
    # Return total regularized cost
    return total_cost
# Set random seed for reproducibility
np.random.seed(10)
# Initialize random weights
initial_w = np.random.rand(X_train.shape[1])
# Initialize bias
initial_b = 0.5
# Set regularization parameter
lambda_ = 0.5
# Compute regularized cost
cost = compute_cost_reg(X_train, y_train, initial_w, initial_b, lambda_
    )
print("Regularized cost :", cost)
```

How it works:

1. Compute cost without regularization (call existing function)
2. Compute regularization term: $\frac{\lambda}{2m} \sum w_j^2$
3. `np.square(w)`: Square each weight
4. `np.sum(...)`: Sum all squared weights
5. Add regularization to base cost

8.2 Regularized Gradients

Gradients with Regularization

Gradient for weights (with regularization):

$$\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})x_j^{(i)} + \frac{\lambda}{m}w_j$$

Gradient for bias (unchanged!):

$$\frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

Key difference: Extra term $\frac{\lambda}{m}w_j$ for weights only!

compute_gradient_reg - FROM INSTRUCTOR'S CODE - INSTRUCTOR'S CODE

File: Session9_Logistic_Regression_Solution.py (lines 856-881)

```python
# UNQ_C6
def compute_gradient_reg(X, y, w, b, lambda_ = 1):
    """
    Computes the gradient for linear regression

    Args:
      X : (ndarray Shape (m,n))   variable such as house size
      y : (ndarray Shape (m,))    actual value
      w : (ndarray Shape (n,))    values of parameters of the model
      b : (scalar)                value of parameter of the model
      lambda_ : (scalar,float)    regularization constant
    Returns
      dj_db: (scalar)             The gradient of the cost w.r.t. the parameter b.
      dj_dw: (ndarray Shape (n,)) The gradient of the cost w.r.t. the parameters w.

    """
    # Get number of examples and features
    m, n = X.shape
    # Compute base gradients without regularization
    dj_db, dj_dw = compute_gradient(X, y, w, b)
    # START CODE HERE
    # Add regularization term to weight gradient
    dj_dw += (lambda_/m)  * w
    # END CODE HERE
    # Return both gradients (bias unchanged)
    return dj_db, dj_dw
# Set random seed for reproducibility
np.random.seed(1)
# Initialize random weights
initial_w  = np.random.rand(X_train.shape[1])
# Initialize bias
initial_b = 0.5
# Set regularization parameter
```

```python
lambda_ = 0.5
# Compute regularized gradients
dj_db, dj_dw = compute_gradient_reg(X_train, y_train,
                                    initial_w, initial_b, lambda_)
print(f"dj_db: {dj_db}", )
print(f"First few elements of regularized dj_dw:\n {dj_dw[:4].tolist()}
    ", )
```

How it works:

1. Compute regular gradients (call existing function)
2. Add regularization term to weight gradient: `(lambda_/m) * w`
3. Bias gradient stays the same (no regularization)
4. Very simple addition!

9 Summary: Key Concepts

9.1 Complete Comparison: Sessions 7, 8, 9

Comparison: Key Differences

Sessions 7, 8, 9 - Complete Comparison

Aspect	Session 7	Session 8	Session 9
Task	Regression (single)	Regression (multiple)	Classification
Output	Continuous number	Continuous number	Probability [0,1]
Model	$wx + b$	$\vec{w} \cdot \vec{x} + b$	$\frac{1}{1+e^{-(\vec{w}\cdot\vec{x}+b)}}$
Features	One (x)	Multiple ($\vec{x}$)	Multiple ($\vec{x}$)
Cost	Squared error	Squared error	Cross-entropy
Gradient	$(f - y)x$	$(f - y)\vec{x}$	$(f - y)\vec{x}$ (but f has sigmoid!)
New concepts	Gradient descent, vectorization	Feature scaling, overfitting	Sigmoid, decision boundaries

What's consistent across all sessions:

- Gradient descent optimization
- Minimize cost function
- Vectorization for speed
- Feature scaling importance
- Regularization to prevent overfitting

9.2 Key Formulas to Memorize

IMPORTANT!

Essential Formulas for Session 9

1. Sigmoid Function:

$$g(z) = \frac{1}{1 + e^{-z}}$$

2. Logistic Regression Model:

$$f_{\vec{w},b}(\vec{x}) = g(\vec{w} \cdot \vec{x} + b) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$$

3. Decision Boundary:

$$\vec{w} \cdot \vec{x} + b = 0$$

Predict $\hat{y} = 1$ if $\vec{w} \cdot \vec{x} + b \geq 0$, else $\hat{y} = 0$

4. Cost Function (Binary Cross-Entropy):

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) \right]$$

5. Regularized Cost:

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(f(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f(\vec{x}^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2$$

6. Gradients:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

7. Regularized Gradients:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)})$$

9.3 Master These Concepts

10 Essential Points

1. **Classification vs Regression**:
   - Regression: Continuous output
   - Classification: Discrete categories (0 or 1)
   - Use logistic regression for classification!
2. **Sigmoid Function**:
   - Maps any number to (0,1)
   - $g(z) = \frac{1}{1+e^{-z}}$
   - Output = probability of $y = 1$
3. **Logistic Regression Model**:
   - Linear combination + sigmoid
   - $f(\vec{x}) = g(\vec{w} \cdot \vec{x} + b)$
   - Output is probability, not class
4. **Decision Boundaries**:
   - Where $\vec{w} \cdot \vec{x} + b = 0$
   - Separates positive from negative class
   - Can be linear or non-linear (with polynomial features)
5. **Binary Cross-Entropy**:
   - Different from squared error!
   - Penalizes confident wrong predictions heavily
   - Makes cost function convex
6. **Gradient Descent**:
   - Same algorithm as linear regression
   - Formula looks identical
   - But $f(\vec{x})$ includes sigmoid!
7. **Predictions**:
   - Compute probability: $f(\vec{x})$
   - Apply threshold: if $f(\vec{x}) \geq 0.5$, predict 1
   - Can adjust threshold based on application
8. **Regularization**:
   - Same concept as Sessions 7-8
   - Add $\frac{\lambda}{2m} \sum w_j^2$ to cost
   - Prevents overfitting in classification

10 Looking Ahead

What's Next?

You now know:

- Linear regression (single and multiple features)
- Logistic regression (binary classification)
- Gradient descent optimization
- Feature engineering and scaling
- Regularization to prevent overfitting
- Complete machine learning pipeline!

Coming soon:

- **Multi-class Classification**: More than 2 categories
- **Neural Networks**: Stack logistic units in layers
- **Deep Learning**: Many layers, complex patterns
- **Advanced Topics**: SVMs, decision trees, ensemble methods

The foundation: Sessions 7, 8, and 9 form the complete foundation for all machine learning!

- Session 7: Basic optimization
- Session 8: Multiple features and overfitting
- Session 9: Classification
- Together: Everything you need to understand modern ML!

10.1 Practical Tips

IMPORTANT!

Common Mistakes to Avoid

1. Using linear regression for classification:

- WRONG! Output not constrained to [0,1]
- Use logistic regression instead

2. Forgetting the sigmoid:

- Must apply sigmoid to get probabilities
- Sigmoid is what makes it logistic regression

3. Using squared error for classification:

- Creates non-convex cost function

- Use binary cross-entropy instead

4. **Not scaling features**:
   - Still important for logistic regression!
   - Use z-score normalization
5. **Only checking training accuracy**:
   - Must check validation accuracy
   - Training accuracy alone doesn't detect overfitting
6. **Setting λ too large**:
   - Causes underfitting
   - Model becomes too simple
   - Start with small λ and increase

11 Conclusion: The Complete Picture

RED THREAD: The Journey from Session 7 → 8 → 9

The Complete Journey:
Session 7: Single feature regression

- One input, one output
- Simple model: $y = wx + b$
- Foundation concepts

Session 8: Multiple feature regression

- Multiple inputs, one output
- Extended model: $y = \vec{w} \cdot \vec{x} + b$
- Feature scaling, overfitting

Session 9: Classification

- Multiple inputs, binary output
- Sigmoid model: $y = g(\vec{w} \cdot \vec{x} + b)$
- Decision boundaries, cross-entropy

The progression:

$$\text{Single feature} \rightarrow \text{Multiple features} \rightarrow \text{Classification}$$

The unifying theme: Gradient descent optimization
You're now ready for:

- Neural networks (many logistic units)
- Deep learning (many layers)
- Advanced machine learning
- Real-world applications!

Master these three sessions - they're the foundation for EVERYTHING in modern AI and machine learning!

12 What You Have ACTUALLY Learned

IMPORTANT!

This section focuses on practical insights directly from your exercises: sigmoid implementation, cost functions, decision boundaries, and especially **regularization** - a major part of the course. Everything here connects to the code you wrote.

From Continuous to Discrete Predictions

The fundamental shift:
Sessions 7-8: Regression

- Output: Any real number (house price: $250,000)
- Question: "How much?"
- Model: $f(\vec{x}) = \vec{w} \cdot \vec{x} + b$

Session 9: Classification

- Output: Probability in [0,1], decision in $\{0, 1\}$
- Question: "Which category?"
- Model: $f(\vec{x}) = \sigma(\vec{w} \cdot \vec{x} + b)$

Key insight: Classification outputs **probabilities**, not categories directly. $f(\vec{x}) = 0.73$ means "73% confident student will be admitted".

12.1 Deep Understanding of Core Concepts

12.1.1 1. The Sigmoid Function

Why Sigmoid Works

What you implemented:

$$g(z) = \frac{1}{1 + e^{-z}}$$

Why this function?
1. Probabilistic Interpretation:
The sigmoid converts **log-odds** to **probability**!
If odds $= \frac{p}{1-p}$, then $\log(\text{odds}) = z$
Solving for p: $p = \frac{1}{1+e^{-z}}$ (the sigmoid!)
What this means: Your linear model $z = \vec{w} \cdot \vec{x} + b$ predicts log-odds. Sigmoid converts to probability.
2. Key Properties You Use:

- **Range**: Always outputs $(0, 1)$ - valid probabilities!
- **Smooth**: Differentiable everywhere - gradient descent works
- **Interpretable**: $g(0) = 0.5$ (uncertain), $g(\infty) \to 1$, $g(-\infty) \to 0$

3. The Decision Boundary Connection:
$g(z) = 0.5 \Leftrightarrow z = 0 \Leftrightarrow \vec{w} \cdot \vec{x} + b = 0$
The boundary where your model is exactly uncertain!

12.1.2 2. Cross-Entropy Loss

Why Not Squared Error?

You learned: Don't use $(f - y)^2$ for classification!
The problem: Squared error + sigmoid = non-convex cost (multiple local minima)
The solution: Binary cross-entropy

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log(f(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f(\vec{x}^{(i)})) \right]$$

Why it works:
1. Convexity: Has single global minimum (gradient descent succeeds!)
2. Gradient Simplicity:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} (f(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

Looks identical to linear regression, but f includes sigmoid! The derivative of sigmoid and log perfectly cancel.
3. Penalizes Confident Mistakes:

- Predict $f = 1$ when $y = 1$: Loss $= 0$ (correct!)
- Predict $f = 0.01$ when $y = 1$: Loss $\to \infty$ (heavily penalized!)

This forces the model to be confident only when justified by data.

12.1.3 3. Decision Boundaries

Geometric View of Classification

What you visualized: The line/curve separating admitted from rejected students
Mathematical definition: Where $\vec{w} \cdot \vec{x} + b = 0$
Key insights:
1. The weight vector $\vec{w}$ is perpendicular to the boundary
Why? On the boundary, $\vec{w} \cdot \vec{x} + b = 0$. Moving along the boundary doesn't change this value, so $\vec{w} \cdot \Delta\vec{x} = 0$ (perpendicular!)
2. Distance from boundary = confidence
For point $\vec{x}$:

$$\text{signed distance} = \frac{\vec{w} \cdot \vec{x} + b}{||\vec{w}||}$$

- Large positive distance: Confident $y = 1$
- Near zero: Uncertain ($p \approx 0.5$)
- Large negative: Confident $y = 0$

3. Linear vs Non-linear Boundaries

- Basic model: Linear boundary (straight line/hyperplane)
- With polynomial features: Curved boundaries (circles, ellipses)

Example: Adding x_1^2, x_2^2 allows circular boundaries!

12.2 Regularization: The Critical Addition

IMPORTANT!

THIS IS A MAJOR PART OF THE COURSE!
Regularization is essential for real-world classification. Without it, models overfit easily!

12.2.1 1. Why Regularization for Classification?

Overfitting in Logistic Regression

The problem you solved:
With many features or polynomial features, logistic regression can create **overly complex decision boundaries**:

- Training accuracy: Very high (99%+)
- Validation accuracy: Low (much worse!)
- Decision boundary: Wiggly, convoluted, memorizing training data

The cause: Large weights w_j create extreme predictions
When $||\vec{w}||$ is large:

- Small changes in features $\rightarrow$ huge changes in logits
- Sigmoid outputs very close to 0 or 1 (over-confident!)
- Boundary fits noise, not signal

The solution: Penalize large weights!

12.2.2 2. Regularized Cost Function

What You Implemented

Without regularization:

$$J(\vec{w}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(f(\vec{x}^{(i)})) + (1-y^{(i)})\log(1-f(\vec{x}^{(i)}))\right]$$

With regularization (what you coded):

$$J(\vec{w}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(f(\vec{x}^{(i)})) + (1-y^{(i)})\log(1-f(\vec{x}^{(i)}))\right] + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$$

The regularization term: $\frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$
What it does:

- Adds penalty for large weights
- λ controls strength (larger λ = simpler model)
- Bias b is NOT regularized (only weights!)
- Forces smoother decision boundaries

12.2.3 3. Regularized Gradients

Modified Gradient Descent

Gradient for weights (with regularization):

$$\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}(f(\vec{x}^{(i)}) - y^{(i)})x_j^{(i)} + \frac{\lambda}{m}w_j$$

Gradient for bias (unchanged!):

$$\frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(f(\vec{x}^{(i)}) - y^{(i)})$$

What this means in your code:

```
# Without regularization
dj_dw = (1/m) * np.dot(X.T, err)

# With regularization (what you added)
dj_dw = (1/m) * np.dot(X.T, err) + (lambda_/m) * w
#                                    ^^^^^^^^^^^^^^^
#                                    Regularization term!
```

The effect during training:

- Each step: Weights shrink slightly toward zero
- $w := w - \alpha(gradient + \frac{\lambda}{m}w) = w(1 - \frac{\alpha\lambda}{m}) - \alpha \cdot \text{gradient}$
- First term: Weight decay (shrinkage)
- Second term: Standard gradient update

12.2.4 4. Choosing λ

The Regularization Tradeoff

You experimented with different λ values:
$\lambda = 0$ **(no regularization)**:

- Complex boundary, high training accuracy
- May overfit (low validation accuracy)

λ **small (weak regularization)**:

- Slightly smoother boundary
- Small improvement in generalization

λ **moderate (good balance)**:

- Smooth but flexible boundary
- Best validation accuracy ✓

λ **large (strong regularization)**:

- Very simple, almost linear boundary
- Underfits (misses patterns!)

How to choose: Use validation set!

1. Try $\lambda \in \{0, 0.01, 0.1, 1, 10, 100\}$
2. For each, train model and measure validation accuracy
3. Pick λ with highest validation accuracy

12.2.5 5. L2 vs L1 Regularization

What You Used (L2)

L2 regularization (Ridge) - what you implemented:

$$\text{Penalty} = \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2$$

Properties:

- Shrinks all weights toward zero
- Weights never exactly zero (keeps all features)
- Smooth, differentiable
- Most commonly used

L1 regularization (Lasso) - alternative:

$$\text{Penalty} = \frac{\lambda}{2m} \sum_{j=1}^{n} |w_j|$$

Properties:

- Some weights become exactly zero
- Performs feature selection automatically
- Less common in deep learning

When to use which:

- L2: Default choice, works well generally
- L1: When you have many irrelevant features and want automatic selection

12.3 Connection to Neural Networks

The Building Block You've Mastered

Key revelation: Logistic regression IS a 1-layer neural network!
What you implemented:

$$\text{Input } \vec{x} \to \text{Linear: } z = \vec{w} \cdot \vec{x} + b \to \text{Sigmoid: } a = \sigma(z) \to \text{Output}$$

A neural network:

$$\text{Input} \to [\text{Many logistic units}] \to [\text{Many logistic units}] \to \ldots \to \text{Output}$$

Everything you learned applies:

- Sigmoid activation → Used in hidden layers
- Cross-entropy loss → Used for classification in NNs
- Gradient descent → Backbone of backpropagation
- Regularization → Essential in deep learning (dropout, weight decay)
- Vectorization → Critical for training large networks

Why layers matter:

- 1 layer (you): Linear decision boundaries
- 2+ layers: Non-linear decision boundaries (any shape!)
- Many layers: Learn hierarchical features (edges → objects)

12.4 Practical Lessons from Implementation

What You Discovered Through Coding

1. Probabilistic Output > Hard Classification:
Your `predict` function:

```
f_w = sigmoid(np.dot(X, w) + b)  # Probabilities
p = (f_w >= 0.5)                  # Hard decisions
```

Why probabilities matter:

- Can adjust threshold (0.3 for medical, 0.7 for spam)
- Rank predictions by confidence
- Flag uncertain cases ($p \approx 0.5$) for review

2. Train/Validation Split is Essential:
Your code splits 66% train, 34% validation:

- Train: Learn parameters
- Validation: Detect overfitting
- If train accuracy >> validation: Overfitting!

3. Feature Scaling Still Matters:
You used `zscore_normalize_features`:

- Exam scores on different scales → slow convergence
- After normalization → faster, more stable training
- Always normalize for production ML!

4. Vectorization is Not Optional:
Compare:

```
# Slow: loops
for i in range(m):
    for j in range(n):
        dj_dw[j] += err[i] * X[i,j]

# Fast: vectorized (what you used)
dj_dw = (1/m) * np.dot(X.T, err)
```

10-100x speedup for real datasets!
5. Monitoring Multiple Metrics:
Your `gradient_descent_val` tracked:

- Training cost (should decrease)
- Training accuracy (should increase)
- Validation accuracy (should increase, then plateau)

If validation accuracy drops while training increases: STOP TRAINING (overfitting!)

12.5 Real-World Applications

Where Your Skills Apply

What you can now build:

1. **Medical Diagnosis**:
   - Features: Age, test results, symptoms
   - Output: Disease/No disease
   - Critical: Probability calibration, low threshold (err on caution)
2. **Spam Detection**:
   - Features: Word frequencies, sender info
   - Output: Spam/Not spam
   - Use your logistic regression on word vectors!
3. **Credit Scoring**:
   - Features: Income, history, age
   - Output: Approve/Deny loan
   - Regularization prevents overfitting to training applicants
4. **Customer Churn**:
   - Features: Usage, demographics, support tickets
   - Output: Will cancel/Won't cancel
   - Target high-risk customers for retention

Why logistic regression?

- Fast training and inference
- Interpretable (can explain decisions)
- Works with limited data
- Industry standard baseline

12.6 What You Can Now Explain

IMPORTANT!

Test Your Understanding

1. **Why sigmoid instead of linear model?**
   - *Basic*: Outputs probabilities [0,1]
   - *Deep*: Linear model treats 0,1 as arbitrary numbers. Sigmoid converts log-odds to probabilities. Squared error + sigmoid is non-convex (bad!). Sigmoid is differentiable everywhere (gradient descent works).

2. **What does regularization actually do?**

   - *Basic*: Prevents overfitting
   - *Deep*: Adds $\frac{\lambda}{2m}\sum w_j^2$ to cost, penalizing large weights. Gradient becomes $\frac{\partial J}{\partial w_j} + \frac{\lambda}{m}w_j$, causing weight decay. Shrinks weights each step, preventing over-confident predictions. Smooths decision boundary. Choose λ via validation set.

3. **Why cross-entropy instead of squared error?**

   - *Basic*: Works better for classification
   - *Deep*: Squared error with sigmoid creates non-convex cost (local minima!). Cross-entropy is convex (single minimum). Gradient of cross-entropy + sigmoid is clean: $(f - y)$. Penalizes confident wrong predictions exponentially. Emerges from maximum likelihood estimation.

4. **What's the connection to neural networks?**

   - *Basic*: Neural networks are more complex
   - *Deep*: Logistic regression IS a 1-layer NN. Multi-layer = stack logistic units. Same sigmoid, same cross-entropy, same gradient descent. Backpropagation = chain rule through layers. You've learned the fundamental building block!

5. **When should I use regularization?**

   - *Basic*: When model overfits
   - *Deep*: ALWAYS use some regularization in practice! Start with $\lambda = 0.01$, tune on validation set. Essential with: many features, polynomial features, limited training data. Even deep learning uses regularization (dropout, weight decay, batch norm).

12.7 The Complete Machine Learning Foundation

What You've Accomplished Across All Three Sessions

Session 7: Linear Regression (Single Feature)

- Model: $f(x) = wx + b$
- Optimization: Gradient descent
- Concepts: Cost function, learning rate, vectorization

Session 8: Multiple Linear Regression

- Model: $f(\vec{x}) = \vec{w} \cdot \vec{x} + b$
- New: Feature scaling, engineering, polynomial features
- Challenge: Overfitting, solved by regularization

Session 9: Logistic Regression

- Model: $f(\vec{x}) = \sigma(\vec{w} \cdot \vec{x} + b)$
- New: Sigmoid, cross-entropy, decision boundaries
- Extension: Regularization for classification

The universal framework:

1. Model: $f_\theta(\vec{x})$ (hypothesis)
2. Cost: $J(\theta)$ (how wrong)
3. Optimization: $\theta := \theta - \alpha \nabla J$ (improve)
4. Regularization: Prevent overfitting
5. Validation: Detect overfitting

This framework powers ALL of machine learning!

12.8 Key Takeaways

Master These Lessons

1. **Classification = Probability + Threshold**
   - Model outputs probabilities (0 to 1)
   - Apply threshold to make decisions
   - Can tune threshold for different costs
2. **Sigmoid + Cross-Entropy = Perfect Pair**
   - Sigmoid converts logits to probabilities
   - Cross-entropy creates convex cost function
   - Gradient is clean: $(f - y)$ (looks like linear regression!)
3. **Regularization is Not Optional**
   - Always use in real applications
   - Tune λ on validation set
   - L2 (what you used) shrinks all weights
   - Prevents overfitting without removing features
4. **Decision Boundaries Reveal Understanding**
   - Linear: $\vec{w} \cdot \vec{x} + b = 0$
   - Non-linear: Add polynomial features
   - Regularization smooths boundaries
   - Visualization helps debug
5. **Train/Val Split Detects Problems**
   - Training accuracy alone is meaningless
   - Validation accuracy reveals generalization
   - Gap between them = overfitting
   - Use to choose λ
6. **You've Learned Neural Network Foundations**
   - Logistic regression = 1-layer NN
   - Stack units = deep network
   - Same principles: forward pass, loss, backward pass, update
   - Everything scales!

Congratulations!

You've completed the ML foundations trilogy!

You now understand:

- Gradient descent optimization (Session 7)
- High-dimensional modeling (Session 8)
- Classification & probability (Session 9)
- Regularization to prevent overfitting
- The building blocks of neural networks

These three sessions are the foundation for:

- Neural networks
- Deep learning
- Modern AI systems

You're ready for advanced machine learning!